

# Production Log

---

## Project idea

My first idea was about machine learning and cryptocurrency, but it did not specify an asset or a target. I narrowed it to Bitcoin volatility so that the models could be compared using numerical errors. My research question became:

**To what extent do Random Forest and LSTM justify their additional complexity over rolling historical volatility and GARCH(1,1) when forecasting the next-day update of a volatility proxy constructed from Hyperliquid BTC perpetual-futures returns?**

I chose volatility because it is linked to market risk and can be measured and compared more clearly than a general question about whether Bitcoin will go up or down. My main aim was to find out whether the extra complexity of Random Forest and LSTM was actually useful when compared with simpler statistical methods.

## Planning and research

The early research notes cover log returns, volatility, rolling historical volatility, GARCH, Random Forest and LSTM. The recorded plan uses one Bitcoin dataset and one test period, comparing error together with explanation, running time and whether the result stayed similar in later checks.

The first plan used Yahoo Finance data. The documented source was later changed to Hyperliquid so that the project referred to one specific Bitcoin perpetual-futures market. The data included each day's open, high, low, close, volume and trade count.

The recorded plan used this order:

1. collect and check the Bitcoin data;
2. calculate log returns and a rolling volatility target;
3. build simple baseline methods;
4. add Random Forest and LSTM;
5. compare every model on later data;
6. check the results again using different settings;
7. use the results to answer whether extra complexity was worthwhile.

## Data preparation and first comparison

The saved data retain the original Hyperliquid daily rows alongside the processed dataset. Derived outputs use daily log returns and their 30-day rolling sample standard deviation as the volatility measure. The target is the next day's updated value. Two neighbouring windows share 29 returns, so the task predicts an update, not a completely new future 30-day period.

The comparison starts with rolling historical volatility as a simple, explainable baseline. It adds GARCH to model periods of high and low volatility and lagged linear regression as a simple check before Random Forest and LSTM.

The data remain in date order, with the earlier part used for training and the later part for testing. The comparison does not randomly shuffle the rows because that could allow future market information to affect an earlier forecast. The saved evaluation uses RMSE and MAE on the standard-deviation proxy and QLIKE on squared target and forecast values.

## Problems and changes

Several checks changed the method. One refresh included a daily candle that had not ended, so its close, volume and trade count were incomplete. The data step was changed to keep completed candles only.

The GARCH predictions had also been joined by row number rather than by date. That could compare a forecast with the wrong target day. The matching was changed to use dates, and the full comparison was rerun.

The final method revision corrected the GARCH update order and rolling-target conversion, and LSTM scaling used training data only. The project-local forest was replaced with scikit-learn's Random Forest. Six RF settings and four LSTM settings were compared using the end of the training period; the final test set was not used to choose them. QLIKE, a linear-coefficient table and an LSTM input-removal check were also added.

## Further checks

The saved robustness outputs repeat the comparison with 14-day and 30-day targets, the first and second halves of the test period, low-, medium- and high-volatility periods and four expanding time windows.

Random Forest outputs include the chosen settings, out-of-bag error and feature importance. LSTM outputs cover three seeds and one-input-at-a-time removal to show which inputs most affect its errors. The models use different inputs and transformations, so the result compares the complete tested pipelines; it does not isolate the effect of model type alone.

## Final result

In the final refresh, Hyperliquid returned 1,241 daily rows. Excluding one unfinished row left 1,240 completed days through 2026-07-19. The prepared data use 950 earlier rows for training and 245 later rows for testing.

The final 30-day results were:

| Model                         | RMSE       | QLIKE      |
|-------------------------------|------------|------------|
| GARCH(1,1)                    | 0.00098502 | 0.00370047 |
| Lagged linear regression      | 0.00140087 | 0.00623445 |
| Rolling historical volatility | 0.00142744 | 0.00669043 |
| LSTM                          | 0.00174351 | 0.00816805 |
| Random Forest                 | 0.00220594 | 0.01170773 |

GARCH ranked first in the main comparison and also remained first in the 14-day and expanding-window checks. Neither LSTM nor Random Forest beat the rolling baseline overall.

Within this comparison, GARCH performed best. Random Forest and LSTM were more complicated and did not beat the rolling baseline. The finding is limited to the tested market, dates, inputs and pipelines; a different dataset or design may give another ranking.

## AI and authorship note

OpenAI Codex assisted with code review and editing, data processing and checks, software tests, writing and editing, English-Chinese translation, and DOCX/PDF export. Repository files and timestamps show saved work, not who performed each action. Only research choices and checks personally verified by the candidate should be presented as the candidate's own work.

## Reflection

The saved corrections show that date alignment, completed-candle filtering and information timing affected the comparison. The result is limited to the tested market, dates, features and pipelines.

**Candidate adds own reflection:** {{CANDIDATE\_REFLECTION\_IN\_OWN\_WORDS}}